



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 9

Team No.: 23

Project Title: **Multi-Agent AI Execution and Resource Management System**

Team Members

Roll Number	Student Name	Signature
2520030154	P. Bhavana Manaswini	
2520030367	k. Rishitha	

1. Abstract

Multi-Agent AI Execution and Resource Management System is a Linux-based Operating Systems project that demonstrates how multiple AI agents can be created, executed, coordinated, and managed as independent processes. In a multi-agent environment, several agents may need to perform different tasks simultaneously, making process management, resource allocation, communication, and synchronization important.

The proposed system uses Operating System concepts to manage multiple AI agents and their execution. Each agent can be treated as a separate process responsible for a particular task. A central manager process can create agents, assign tasks, monitor their execution, and control their resource usage.

The project focuses on key Operating Systems and system programming concepts such as **process creation, process identification, inter-process communication (IPC), process scheduling, synchronization, resource management, and process termination**. Linux system calls such as `fork()`, `getpid()`, `wait()`, and IPC mechanisms can be used to implement the system.

The expected outcome is a functional Linux-based application that demonstrates how multiple AI agents can execute tasks while the system manages their processes and available resources efficiently.

2. Problem Statement

In a multi-agent AI system, several agents may need to execute different tasks at the same time. Without proper process and resource management, multiple agents can compete for CPU, memory, and other system resources, resulting in inefficient or uncoordinated execution.

This project addresses the problem of managing the execution of multiple AI agents using Operating System concepts. The system demonstrates how a manager process can create and control multiple agent processes, assign tasks, monitor their status, and coordinate their execution.

The proposed system uses Linux processes and system programming mechanisms to demonstrate communication, synchronization, scheduling, and resource management between the agents.

The project is relevant to Operating Systems and Systems Programming because it provides a practical demonstration of **process management, IPC, synchronization, scheduling, and resource allocation** in a multi-agent environment.

3. Objectives

- To understand how multiple AI agents can be represented and managed as processes.
- To create and manage multiple processes using the `fork()` system call.
- To identify and monitor individual agent processes using PID and PPID.
- To demonstrate communication between the manager process and agent processes using IPC mechanisms.
- To implement task assignment and execution among multiple agents.
- To demonstrate process synchronization so that agents can coordinate their execution.
- To monitor and manage system resources used by different agent processes.
- To demonstrate safe completion and termination of agent processes

4. Proposed Methodology

The project will be designed and implemented as a Linux-based C/C++ application that demonstrates multi-agent execution and resource management using Operating System concepts.

1. Agent Creation

The main or manager process will create multiple agent processes using the `fork()` system call.

Example:

- Agent 1 → Data Processing
- Agent 2 → Task Analysis
- Agent 3 → Resource Monitoring
- Agent 4 → Result Processing

Each agent will perform its assigned task independently.

2. Process Identification

Each agent process will obtain its Process ID (PID) and Parent Process ID (PPID) using functions such as `getpid()` and `getppid()`.

This allows the manager to identify and monitor individual agents.

3. Task Assignment : The manager process will assign different tasks to the available agents.

The system can maintain a task queue where pending tasks are stored and assigned to available agents.

4. Agent Execution : Each agent process performs its assigned task independently.

The manager monitors the execution status of the agents and coordinates their activities.

5. Inter-Process Communication : The manager and agents will communicate using suitable IPC mechanisms.

IPC can be used to:

- Send tasks to agents
- Send results back to the manager
- Exchange status information
- Coordinate agent execution

6. Resource Management : The system will monitor the resources required by different agent processes.

The resource manager can consider resources such as:

- CPU usage
- Memory usage
- Number of active processes
- Task workload

The manager can coordinate execution so that resources are not unnecessarily consumed by multiple agents.

7. Synchronization : Synchronization mechanisms will be used to coordinate agents and prevent conflicts during shared operations.

The manager can use mechanisms such as wait() to monitor child processes and ensure proper completion.

8. Process Control The manager process will control the execution of the agents.

It can:

- Start agents
- Assign tasks
- Monitor agents
- Wait for completion
- Handle failures
- Terminate completed processes

9. Execution Flow

Start Manager → Create Agent Processes → Assign Tasks → Execute Tasks → Communicate Results → Monitor Resources → Synchronize Agents → Complete/Terminate Processes

10. Testing

The application will be executed in a Linux terminal and tested with multiple agents and different task workloads to verify process creation, communication, synchronization, and resource management.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Creation – fork()	Creates multiple agent processes from the manager process.
Process Identification – getpid()	Retrieves the PID of each agent process.
Parent Identification – getppid()	Identifies the parent/manager process of an agent.
Inter-Process Communication (IPC)	Allows the manager and agents to exchange tasks, results, and status information.
Process Synchronization – wait()	Allows the manager to wait for agent processes and ensures proper execution.
Process Scheduling	Determines how multiple agent processes get CPU execution time.
Resource Management	Monitors and manages resources used by agent processes.
Process Termination	Ensures completed or unnecessary agent processes are safely terminated.
Linux /proc	Can be used to observe information about running processes and resource usage.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility

7. Tools / Platforms / Software Used

Tool / Platform/ Software	Purpose
Linux / Ubuntu	Provides the operating environment for developing and executing the project.
C / C++	Used to implement processes, IPC, synchronization and resource management.
GCC / G++ Compiler	Compiles the C/C++ source code.
POSIX Libraries	Provides APIs and functions for signals, processes, and synchronization.
Linux Terminal / Shell	Used to compile, execute, test, and monitor processes and signals.

8. Expected Outcome

The project is expected to produce a functional Linux-based multi-agent execution and resource management application.

The expected outcomes are:

1. The system will successfully create multiple agent processes using fork().
2. Each agent will display its PID and PPID for identification and monitoring.
3. The manager process will assign different tasks to different agents.
4. Agents will execute their assigned tasks independently.
5. The manager and agents will communicate using suitable IPC mechanisms.
6. The system will demonstrate synchronization between multiple agent processes.
7. Resource usage of the agent processes can be monitored and managed.
8. The system will demonstrate safe completion and termination of agent processes.
9. The terminal output will clearly show agent creation, task assignment, execution, communication, resource monitoring and process termination.
10. Overall, the project will provide practical understanding of **Linux process management, IPC, synchronization, scheduling and resource management** in a multi-agent AI environment.

9. GitHub Repository

GitHub Repository Link:

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name:

Signature:

Remarks:

Faculty Use

Project Approved:

☐ Yes ☐ No

Date:

Faculty Signature:

